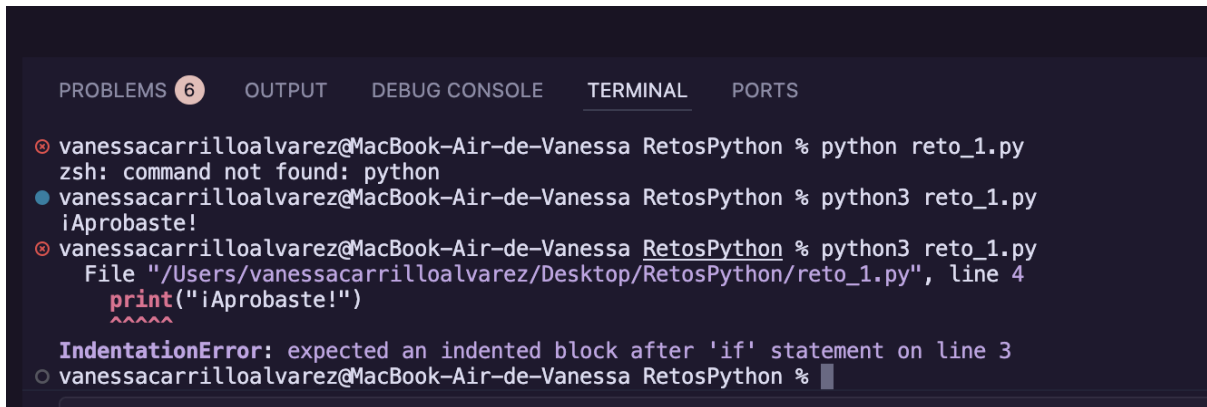


PYTHON DESDE CERO

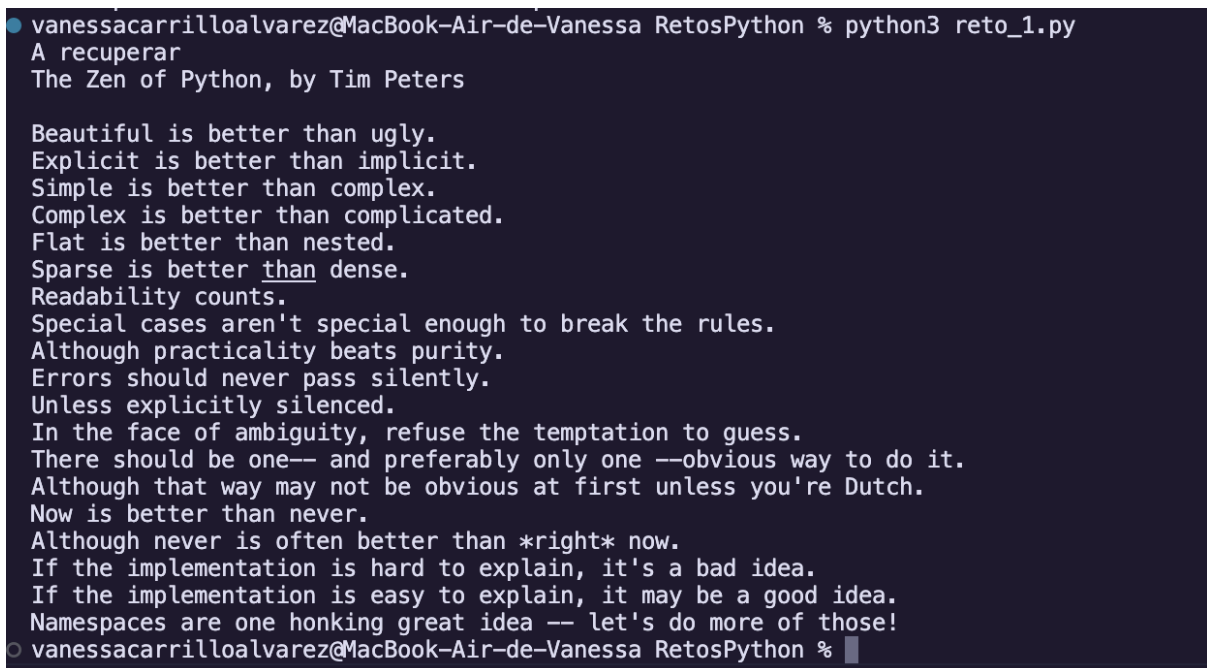
RETO DE OBSERVADOR 1

2.



```
PROBLEMS 6 OUTPUT DEBUG CONSOLE TERMINAL PORTS
vanessacarrilloalvarez@MacBook-Air-de-Vanessa RetosPython % python reto_1.py
zsh: command not found: python
vanessacarrilloalvarez@MacBook-Air-de-Vanessa RetosPython % python3 reto_1.py
¡Aprobaste!
vanessacarrilloalvarez@MacBook-Air-de-Vanessa RetosPython % python3 reto_1.py
File "/Users/vanessacarrilloalvarez/Desktop/RetosPython/reto_1.py", line 4
    print("¡Aprobaste!")
    ^^^^^
IndentationError: expected an indented block after 'if' statement on line 3
vanessacarrilloalvarez@MacBook-Air-de-Vanessa RetosPython %
```

3.



```
vanessacarrilloalvarez@MacBook-Air-de-Vanessa RetosPython % python3 reto_1.py
A recuperar
The Zen of Python, by Tim Peters

Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -- let's do more of those!
vanessacarrilloalvarez@MacBook-Air-de-Vanessa RetosPython %
```

"If the implementation is hard to explain, it's a bad idea."

Significa que el buen código debe ser simple y directo, si una solución resulta demasiado compleja de explicar a otros desarrolladores, probablemente existan formas más limpias y eficientes de resolverlo.

4.

- **Netflix:** Lo utiliza para automatizar la gestión de su infraestructura de red, analizar el comportamiento de reproducción de los usuarios y alimentar sus algoritmos de recomendación de contenido.

- **Instagram:** Ejecuta la lógica del servidor y la plataforma de gestión de fotos/videos sobre un motor construido en Python.
- **Spotify:** Depende de Python para procesar grandes volúmenes de datos y alimentar el motor de análisis que genera listas personalizadas como *Discover Weekly*.

BONUS

Pixar: Pixar utiliza Python en sus herramientas internas de animación para automatizar la renderización y pipeline de efectos visuales.

CHECKPOINT - NIVEL 1

1. ¿Qué significa que Python sea "interpretado" y qué ventaja tiene para quien aprende?

Respuesta: Significa que el programa lee y ejecuta el código línea por línea en tiempo real, sin tener que transformar todo el archivo en un bloque gigante antes de probarlo, y para aprender es genial porque si se comete un error, se ve al instante, se corrige esa línea y se sigue probando sin perder tiempo.

2. ¿Por qué Python domina en Inteligencia Artificial si por dentro usa C++?

Respuesta: Porque le ahorra vida a los programadores, C++ es rapidísimo pero cansa mucho escribirlo, mientras que Python es fácil de entender pero más lento, entonces unieron los dos, escribe código sencillo en Python y deja que C++ se encargue de la fuerza bruta por detrás.

3. ¿Qué son la indentación y las librerías, y por qué son tan importantes?

Respuesta:

- ❖ **Indentación:** Son los 4 espacios al inicio de las líneas de código, en vez de usar llaves como en otros lenguajes, Python usa estos espacios para saber qué instrucciones van dentro de un **if**, un bucle o una función. Hace que el código se vea limpio y ordenado obligatoriamente.
- ❖ **Librerías:** Son como "cajas de herramientas" o pedazos de código que otros programadores ya escribieron y probaron y en lugar de resolver un problema complejo desde cero, solo importa la librería adecuada y ahorra muchísimo trabajo.

RETO DE OBSERVADOR 2

4.

```
*      *
*      *
*      *
**
2026/08/07
3... 2... 1... ¡Ya!
vanessacarrilloalvarez@MacBook-Air-de-Vanessa RetosPython % python3 retp.py
vanessacarrilloalvarez@MacBook-Air-de-Vanessa RetosPython % python3 retp.py
vanessacarrilloalvarez@MacBook-Air-de-Vanessa RetosPython % python3 reto_2.py
File "/Users/vanessacarrilloalvarez/Desktop/RetosPython/reto_2.py", line 19
    print("Hola, soy Vanessa)
    ^
SyntaxError: unterminated string literal (detected at line 19)
vanessacarrilloalvarez@MacBook-Air-de-Vanessa RetosPython %
```

CHECKPOINT - NIVEL 2

1. ¿Qué diferencia hay entre la función print, sus paréntesis y su argumento?

Respuesta: print es el nombre de la herramienta que ejecuta la acción de mostrar cosas en la consola. Los paréntesis son la puerta de entrada que obligatoriamente le dice a Python que se va a ejecutar esa herramienta. Y el argumento es el dato real que le mete adentro para que lo proyecte en pantalla.

2. ¿Para qué sirven sep y end? Da un ejemplo de cada uno.

Respuesta: Sirven para personalizar cómo se imprime el texto, cuando se le manda varios datos a un print, los separa con un espacio; si se usa sep, puede cambiar ese espacio por lo que quiera, como una barra para que salga 2026/08/07. Por otro lado, end cambia lo que pasa al terminar la línea; en vez de hacer el salto de renglón automático, se puede hacer que se quede en el mismo sitio con print("Cargando...", end=" ") para que lo siguiente que se imprima salga justo al lado.

3. ¿Por qué los comentarios deben explicar el "por qué" y no el "qué"?

Respuesta: Porque lo que hace el código ya se está viendo en la pantalla si sabe leer sintaxis básica. Escribir un comentario para decir "aquí sumo dos números" no le aporta nada a nadie. En cambio, explicar por qué se decidió hacer esa suma o la razón de negocio detrás de esa lógica le ahorra muchísimo tiempo y dolores de cabeza a cualquiera que vuelva a revisar ese proyecto en el futuro.

RETO DE OBSERVADOR 3

1.

```
vanessacarrilloalvarez@MacBook-Air-de-vanessa RetosPython % python3 reto_3.py
vanessacarrilloalvarez@MacBook-Air-de-Vanessa RetosPython % python3 reto_3.py
<class 'str'>
<class 'int'>
<class 'float'>
<class 'bool'>
<class 'str'>
<class 'int'>
vanessacarrilloalvarez@MacBook-Air-de-Vanessa RetosPython %
```

2.

- **x - edad:** La segunda es mejor porque da contexto inmediato sin tener que adivinar qué guarda.
- **h - horas_de_sueño:** Evita confusiones, ya que h podría significar altura, hora o historia.
- **d - dias_del_mes:** La primera es una sola letra que no te dice nada. La segunda te aclara de inmediato que guarda un contador o número de días.

3.

```
# Provocar el error a propósito
print("Tengo " + 17 + " años")

# Forma 1 de arreglarlo: usando str() para convertir los enteros a texto
print("Tengo " + str(17) + " años")

# Forma 2 de arreglarlo: usando comas para separar los argumentos en el print
print("Tengo", 17, "años")

Traceback (most recent call last):
  File "/Users/vanessacarrilloalvarez/Desktop/RetosPython/reto_3.py", line 16, in <module>
    print("Tengo " + 17 + " años")
    ~~~~~^~~~~
TypeError: can only concatenate str (not "int") to str
vanessacarrilloalvarez@MacBook-Air-de-Vanessa RetosPython %
```

4.

¿Qué hace round()?: Toma un número decimal y lo redondea a la cantidad de decimales que se le indique entre los paréntesis (en este caso, a 2 decimales).

¿Por qué resuelve el problema visual?: Limpia el resultado en la consola cortando los decimales sobrantes y dejando un 0.3 presentable.

Por qué no elimina la causa real: El origen del problema es la arquitectura del sistema, las computadoras procesan los datos en binario y no pueden representar de forma exacta ciertas fracciones decimales en base 10 (como

0.1 o 0.2). Por dentro, la máquina sigue guardando esa pequeña imprecisión binaria, `round()` solo maquilla la salida final en pantalla.

CHECKPOINT - NIVEL 3

1. ¿Qué hace exactamente el signo = y por qué es incorrecto leerlo como "igual"?

Respuesta: El signo = no compara si dos cosas son iguales, sino que es un operador de asignación. Lo que hace es tomar el valor que está a la derecha y guardarlo dentro de la variable que está a la izquierda. Decir que es "igual" confunde, porque en programación la comparación de igualdad se hace con un doble signo (==).

2. Nombra los 4 tipos básicos con un ejemplo y una trampa asociada.

Respuesta:

- **str (Texto):** `nombre = "Vane"`. *Trampa:* Poner números entre comillas (como "17"), lo cual hace que Python los trate como texto y no como número.
- **int (Entero):** `edad = 17`. *Trampa:* Intentar hacer operaciones de texto o concatenar directo con un `str` sin convertirlo antes.
- **float (Decimal):** `estatura = 1.65`. *Trampa:* Usar comas en vez de puntos (escribir 1,65), lo que genera un error de sintaxis o crea una tupla.
- **bool (Booleano):** `tiene_mascota = False`. *Trampa:* Escribir `true` o `false` en minúscula; Python exige que la primera letra sea mayúscula (`True / False`).

3. ¿Qué significa que Python sea de tipado dinámico pero fuerte?

Respuesta: Que sea dinámico implica que no hay que especificar el tipo de dato al crear una variable, el lenguaje lo detecta solo y permite asignarle un dato distinto más adelante sin poner problema y que sea fuerte significa que no tolera mezclar tipos incompatibles por accidente (como intentar sumar un texto con un número). Si no se convierte el dato manualmente primero la ejecución se detiene y marca error para evitar datos corruptos.

RETO DE OBSERVADOR 4

1.

```
1  nota1 = float(input("Ingresa la primera nota: "))
2  nota2 = float(input("Ingresa la segunda nota: "))
3  nota3 = float(input("Ingresa la tercera nota: "))
4
5  promedio = (nota1 + nota2 + nota3) / 3
6
7  print(f"El promedio final es: {promedio:.2f}")
```

```
vanessacarrilloalvarez@MacBook-Air-de-Vanessa RetosPython % python3 reto_4.py
Ingresa la primera nota: 4.5
Ingresa la segunda nota: 3.0
Ingresa la tercera nota: 2.3
El promedio final es: 3.27
vanessacarrilloalvarez@MacBook-Air-de-Vanessa RetosPython %
```

2.

```
nombre = input("Ingresa el nombre: ")
edad = int(input("Ingresa la edad: "))
ciudad = input("Ingresa la ciudad: ")

año_cumple_30 = 2026 + (30 - edad)

print(f"Hola, mi nombre es {nombre}, vivo en {ciudad} y tengo {edad} años. En el año {2026 + (30 - edad)} cumpliré 30 años.")
```

```
Ingresa el nombre: Vanessa
Ingresa la edad: 18
Ingresa la ciudad: Armenia
Hola, mi nombre es Vanessa, vivo en Armenia y tengo 18 años. En el año 2038 cumpliré 30 años.
vanessacarrilloalvarez@MacBook-Air-de-Vanessa RetosPython %
```

3.

Al escribir "hola":

Muestra el error

```
vanessacarrilloalvarez@MacBook-Air-de-Vanessa RetosPython % python3 reto_4.py
Ingresa la primera nota: hola
Traceback (most recent call last):
  File "/Users/vanessacarrilloalvarez/Desktop/RetosPython/reto_4.py", line 1, in <module>
    nota1 = float(input("Ingresa la primera nota: "))
ValueError: could not convert string to float: 'hola'
vanessacarrilloalvarez@MacBook-Air-de-Vanessa RetosPython %
```

Ocurre porque la función `float()` espera recibir un texto compuesto por caracteres numéricos. Al enviarle letras, no puede realizar la conversión matemática y detiene el programa.

Al escribir "4,5" (con coma):

Muestra el error

```

vanessacarrilloalvarez@MacBook-Air-de-Vanessa RetosPython % python3 reto_4.py
Ingresa la primera nota: 4,5
Traceback (most recent call last):
  File "/Users/vanessacarrilloalvarez/Desktop/RetosPython/reto_4.py", line 1, in <module>
    nota1 = float(input("Ingresa la primera nota: "))
ValueError: could not convert string to float: '4,5'
vanessacarrilloalvarez@MacBook-Air-de-Vanessa RetosPython %

```

En sintaxis de programación Python, los números decimales se definen estrictamente con punto (.). La coma es un separador de sintaxis (usado para listas o argumentos), por lo que `float()` no la reconoce como parte de una cifra decimal.

4.

```

1  # Encabezados
2  print(f"{'Nombre':<15} | {'Edad':^5} | {'Nota':>6}")
3  print("-" * 32)
4
5  # Filas de estudiantes
6  print(f"{'Vane':<15} | {17:^5} | {4.8:>6.1f}")
7  print(f"{'Alejandro':<15} | {18:^5} | {4.2:>6.1f}")
8  print(f"{'Santiago':<15} | {19:^5} | {3.9:>6.1f}")

```

```

vanessacarrilloalvarez@MacBook-Air-de-Vanessa RetosPython % python3 reto_4.py
Nombre      | Edad  |  Nota
-----
Vane        |    17 |  4.8
Alejandro   |    18 |  4.2
Santiago    |    19 |  3.9
vanessacarrilloalvarez@MacBook-Air-de-Vanessa RetosPython %

```

CHECKPOINT - NIVEL 4

1. ¿Por qué `int(input())` es tan común? ¿Qué hace cada función y en qué orden se ejecutan?

Respuesta: Es muy común porque la función `input()` siempre devuelve los datos capturados en formato de texto (`str`), sin importar si se escriben números, y si se necesita hacer operaciones matemáticas, es obligatorio convertir ese texto a un número entero.

Las funciones se ejecutan de adentro hacia afuera:

- Primero se ejecuta `input()`, muestra el mensaje en pantalla, espera a que se ingrese un dato en la consola y lo recibe como texto.

- Luego se ejecuta `int()`, toma ese texto devuelto por `input()` y lo transforma en un valor de tipo entero para poder operarlo numéricamente.

2. Explica por qué "5" + "3" da "53" y cómo se arregla.

Respuesta: Da "53" porque al estar entre comillas, Python interpreta ambos valores como cadenas de texto. En los textos, el operador + no realiza una suma matemática, sino una concatenación. Se arregla quitando las comillas o convirtiendo los valores a enteros con `int()` antes de sumar:

3. Escribe de memoria una f-string que muestre un promedio con dos decimales.

Respuesta: `print(f"El promedio es: {promedio:.2f}")`

RETO DE OBSERVADOR 5

1.

```
vanessacarrilloalvarez@MacBook-Air-de-Vanessa RetosPython % python3 reto_5.py
Ingresa el primer número: 17
Ingresa el segundo número: 5
Suma: 22.00
Resta: 12.00
Multiplicación: 85.00
División: 3.40
División entera: 3.00
Módulo (residuo): 2.00
Potencia: 1419857.00
vanessacarrilloalvarez@MacBook-Air-de-Vanessa RetosPython %
```

2.

```

1  num = int(input("Ingresa un número entero: "))
2
3  es_par = num % 2 == 0
4  es_multiplo_3 = num % 3 == 0
5  en_rango = 1 <= num <= 100
6
7  print(f"¿Es par?: {es_par}")
8  print(f"¿Es múltiplo de 3?: {es_multiplo_3}")
9  print(f"¿Está entre 1 y 100?: {en_rango}")

```

```

● vanessacarrilloalvarez@MacBook-Air-de-Vanessa RetosPython % python3 reto_5.py
Ingresa un número entero: 20
¿Es par?: True
¿Es múltiplo de 3?: False
¿Está entre 1 y 100?: True
○ vanessacarrilloalvarez@MacBook-Air-de-Vanessa RetosPython %

```

3.

```

1  precio = 80000
2  talla = "M"
3  color = "negro"
4  hay_stock = True
5
6  # Expresión única para validar el filtro
7  cumple_busqueda = precio <= 100000 and talla == "M" and color == "negro" and hay_stock
8
9  print(f"¿El producto cumple la búsqueda?: {cumple_busqueda}")

```

4. Predicciones:

- $10 / 2$ va a dar 5.0 (Si dió 5.0 porque en Python la división normal da un float).
- $10 // 3$ va a dar 3 (Si dio 3 porque descarta los decimales y se queda solo con la parte entera).
- $2 ** 3 ** 2$ va a dar 64 (No dió 64, sino 512).

¿Por qué $2 ** 3 ** 2$ da 512?

Porque la mayoría de operadores en Python se evalúan de izquierda a derecha, pero la potencia ($**$) se evalúa de derecha a izquierda (asociatividad por la derecha). Primero resuelve $3^2 = 9$, y luego calcula $2^9 = 512$. Si fuera de izquierda a derecha daría $(2^3)^2 = 8^2 = 64$.

CHECKPOINT - NIVEL 5

1. Diferencia entre $/$, $//$ y $%$ con ejemplo

Respuesta:

- ❖ / (División decimal): Divide dos números y devuelve el resultado con decimales (float).
Ejemplo: 7 / 2 da 3.5.
- ❖ // (División entera): Divide dos números y descarta la parte decimal, devolviendo solo el entero redondeado hacia abajo.
Ejemplo: 7 // 2 da 3.
- ❖ % (Módulo): Devuelve el residuo o sobrante de una división entera.
Ejemplo: 7 % 2 da 1 (porque 2 cabe 3 veces en 7 y sobra 1).

2. Diferencia entre = y == y por qué es el error más común

Respuesta:

- ❖ = (Asignación): Se usa para guardar un valor dentro de una variable (edad = 17).
- ❖ == (Comparación): Se usa para preguntar si dos valores son iguales y devuelve True o False (edad == 17).

Es el error más común porque la costumbre matemática lleva a usar un solo = para comparar, y en programación, usar = dentro de una condición causa un error de sintaxis al intentar asignar un valor donde se esperaba evaluar una condición.

3. Traducción a español de: edad >= 18 and (tiene_entrada or es_socio)

Respuesta: "Tiene al menos 18 años Y (cuenta con entrada o es socio)."

RETO DE OBSERVADOR 6

1.

```

reto_6.py > ...
1  edad = int(input("Ingresa tu edad: "))
2
3  if edad < 0:
4      print("Edad no válida")
5  elif edad <= 11:
6      print("Niño")
7  elif edad <= 17:
8      print("Adolescente")
9  elif edad <= 64:
10     print("Adulto")
11 else:
12     print("Adulto mayor")

```

```

vanessacarrilloalvarez@MacBook-Air-de-Vanessa RetosPython % python3 reto_6.py
Ingresa tu edad: 11
Niño
vanessacarrilloalvarez@MacBook-Air-de-Vanessa RetosPython % python3 reto_6.py
Ingresa tu edad: 56
Adulto
vanessacarrilloalvarez@MacBook-Air-de-Vanessa RetosPython %

```

2.

```

reto_6.py > ...
1  temperatura = float(input("Ingresa la temperatura: "))
2
3  # Ternario guardado en variable
4  mensaje = "Hace calor" if temperatura > 25 else "Está fresco"
5  print(mensaje)
6
7  # Directo dentro de una f-string
8  print(f"El clima hoy: {'Hace calor' if temperatura > 25 else 'Está fresco'}")

```

```

vanessacarrilloalvarez@MacBook-Air-de-Vanessa RetosPython % python3 reto_6.py
Ingresa la temperatura: 34
Hace calor
El clima hoy: Hace calor
vanessacarrilloalvarez@MacBook-Air-de-Vanessa RetosPython % python3 reto_6.py
Ingresa la temperatura: 12
Está fresco
El clima hoy: Está fresco
vanessacarrilloalvarez@MacBook-Air-de-Vanessa RetosPython %

```

3.

```
reto_6.py > ...
1
2  print("---- MENÚ DE OPCIONES ----")
3  print("1. Consultar nota")
4  print("2. Ver promedio")
5  print("3. Ayuda")
6  print("4. Salir")
7
8  opcion = input("Elige una opción (1-4): ")
9
10 match opcion:
11     case "1":
12         print("Consultando tu nota...")
13     case "2":
14         print("Calculando tu promedio...")
15     case "3":
16         print("Abriendo la sección de ayuda...")
17     case "4":
18         print("Saliendo del programa...")
19     case _:
20         print("Opción no válida. Por favor elige entre 1 y 4.")

vanessacarrilloalvarez@MacBook-Air-de-Vanessa RetosPython % python3 reto_6.py
---- MENÚ DE OPCIONES ----
1. Consultar nota
2. Ver promedio
3. Ayuda
4. Salir
Elige una opción (1-4): 2
Calculando tu promedio...
vanessacarrilloalvarez@MacBook-Air-de-Vanessa RetosPython %
```

4.

```
nota = 5.0

if nota >= 3.0:
    print("Aprobado")
elif nota >= 4.5:
    print("Excelente")
```

❖ Se asigna la variable `nota = 5.0`.

- ❖ Se evalúa el primer bloque: `if 5.0 >= 3.0`.
- ❖ Como esa condición se cumple (True), el programa ejecuta `print("Aprobado")`.
- ❖ Al encontrar una condición verdadera, Python ignora el resto de los `elif` y sale de la estructura. El código finaliza mostrando "Aprobado", ocultando el resultado "Excelente" que correspondía a esa nota.

Regla violada

Las condiciones en un bloque `if/elif` se deben evaluar siempre desde el caso más específico o restrictivo hasta el más general.

CHECKPOINT - NIVEL 6

1. ¿Por qué la indentación en Python no es estética? Da un ejemplo de cómo cambia el significado.

Respuesta: La indentación no es estética porque en Python reemplaza las llaves (`{}`) de otros lenguajes para definir la estructura del código. Sangrar una línea le indica explícitamente al intérprete qué instrucciones pertenecen al bloque interno de un `if`, `elif`, `else` o función, y cuáles están fuera.

Si se escribe un `if` que valida si un usuario inició sesión, y en la línea de abajo se coloca la instrucción de mostrar el panel principal con sangría, la pantalla solo se abrirá cuando el inicio de sesión sea exitoso. Si esa misma línea de mostrar el panel se coloca sin sangría, Python asume que es una instrucción independiente fuera del `if`, por lo que el panel se abrirá siempre, incluso si el usuario no inició sesión.

Esa diferencia de un tabulador es la que cambia completamente el comportamiento del programa.

2. ¿Por qué el orden de los `elif` es tan importante? Explica con la cadena de notas.

Respuesta: Porque el programa evalúa las condiciones en cascada de arriba hacia abajo y se detiene apenas encuentra la primera que dé verdadero, ignorando todo lo demás. En el sistema de notas, si se evalúa primero si la nota es mayor o igual a 3.0 para marcar "Aprobado", una nota de 5.0 encajará de una en esa regla y el programa nunca llegará a revisar la condición de si es mayor a 4.5 para marcar "Excelente".

3. ¿Cuándo conviene `match` en vez de una cadena de `elif`, y cuándo no?

Respuesta: Conviene cuando se necesita comparar el valor exacto de una misma variable contra varias alternativas fijas (como las opciones 1, 2 o 3 de un menú). En cambio, no resulta cómodo de usar cuando se requiere evaluar rangos continuos (como saber si una cifra está entre 10 y 20) o cuando intervienen condiciones lógicas combinadas con `and` o `or`; ahí una estructura tradicional con `if` y `elif` resulta mucho más clara y flexible.

RETO DE OBSERVADOR 7

1.

```

● vanessacarrilloalvarez@MacBook-Air-de-Vanessa RetosPython % python3 reto_7.py
=====
BOLETÍN DE CALIFICACIONES
=====
Nombre del estudiante: Vanessa
Nota 1: 5.0
Nota 2: 4.8
Nota 3: 4.5

Estudiante: Vanessa
Promedio: 4.77
Mejor nota: 5.0
Estado: APROBADO
Desempeño: SUPERIOR 🏆

¿Qué quieres hacer? (repasar / mejorar / salir)
> mejorar
Para llegar a 4.0 te faltan -2.3 puntos en total.
● vanessacarrilloalvarez@MacBook-Air-de-Vanessa RetosPython % python3 reto_7.py
=====
BOLETÍN DE CALIFICACIONES
=====
Nombre del estudiante: Vanessa
Nota 1: 3.0
Nota 2: 2.4
Nota 3: 1.5

Estudiante: Vanessa
Promedio: 2.30
Mejor nota: 3.0
Estado: REPROBADO
Desempeño: BAJO

¿Qué quieres hacer? (repasar / mejorar / salir)
> repasar
Empieza por la materia de tu nota más baja.
● vanessacarrilloalvarez@MacBook-Air-de-Vanessa RetosPython % python3 reto_7.py
=====
BOLETÍN DE CALIFICACIONES
=====
Nombre del estudiante: Vanessa
Nota 1: 3.0
Nota 2: 3.0
Nota 3: 3.0

Estudiante: Vanessa
Promedio: 3.00
Mejor nota: 3.0
Estado: APROBADO
Desempeño: BÁSICO

¿Qué quieres hacer? (repasar / mejorar / salir)
> mejorar
Para llegar a 4.0 te faltan 3.0 puntos en total.
○ vanessacarrilloalvarez@MacBook-Air-de-Vanessa RetosPython % █

```

2.

```

● vanessacarrilloalvarez@MacBook-Air-de-Vanessa RetosPython % python3 reto_7.py
=====
BOLETÍN DE CALIFICACIONES
=====
Nombre del estudiante: vanessa
Nota 1: 5
Nota 2: 4
Nota 3: 3

Estudiante: Vanessa
Promedio: 4.00
Mejor nota: 5.0
Estado: APROBADO
Desempeño: ALTO

¿Qué quieres hacer? (repasar / mejorar / salir)
> mejorar
¡Ya superaste la meta de 4.0! No necesitas puntos adicionales.
● vanessacarrilloalvarez@MacBook-Air-de-Vanessa RetosPython % python3 reto_7.py

```

3. Las dos funcionalidades que se agregaron al programa son:

- ❖ **Nuevo case "beca" en la estructura match:** Añadí una opción adicional en el menú que toma el promedio calculado y le muestra al usuario si califica para el 100% de beca académica, un 50% de descuento o los requisitos para postularse.
- ❖ **Mensajes dinámicos por rango de nota:** Modifiqué los bloques de decisiones if/elif/else agregando consejos y recomendaciones académicas personalizadas según el desempeño alcanzado (Superior, Alto, Básico o Bajo).

4.

- ❖ Error de sintaxis (SyntaxError):
 - *Error provocado:* Olvidar los dos puntos : al final del if promedio >= 4.5.
 - *Qué salió:* SyntaxError: expected ':'.
 - *Cómo se detectó:* Python detuvo la ejecución inmediatamente antes de correr el programa y marcó la línea con una flecha.
 - *Solución:* Agregar los dos puntos : al final de la instrucción condicional.
- ❖ Error de ejecución (ValueError):
 - *Error provocado:* Escribir "cuatro" en lugar de 4.0 cuando pide la nota.

- *Qué salió:* `ValueError: could not convert string to float: 'cuatro'.`
 - *Cómo se detectó:* El programa arrancó bien, pero se detuvo abruptamente al llegar al `float(input())`.
 - *Solución:* Asegurarse de ingresar únicamente valores numéricos compatibles con `float()`.
- ❖ Error lógico:
- *Error provocado:* Calcular el promedio sin paréntesis: `promedio = n1 + n2 + n3 / 3`.
 - *Qué salió:* No genera ningún mensaje de error en consola, pero da un promedio erróneo (por ejemplo, `10.13`).
 - *Cómo se detectó:* Mediante una prueba de escritorio calculando el valor real a mano.
 - *Solución:* Agregar los paréntesis `(n1 + n2 + n3) / 3` para forzar a que la suma se ejecute antes que la división.

CHECKPOINT - NIVEL 7

1. ¿Cuál de los 3 tipos de error es más peligroso y por qué Python no puede ayudarte con él?

Respuesta: El error lógico es el más peligroso. Python no puede advertir sobre él ni detener el programa porque la sintaxis es totalmente válida y el código ejecuta sin romperse; sin embargo, entrega resultados incorrectos que solo el desarrollador puede detectar revisando la lógica.

2. Explica cómo se lee un traceback y qué línea contiene la información más útil.

Respuesta: Un traceback se lee de abajo hacia arriba. La información más útil se encuentra en la última línea, ya que indica el tipo exacto de error y una descripción breve del problema, mientras que la línea inmediatamente superior especifica el archivo y el número de línea donde ocurrió.

3. En el bug del promedio, ¿qué regla del Nivel 5 se había violado y cómo se arregla?

Respuesta: Se había violado la precedencia de los operadores. Se arregla utilizando paréntesis para agrupar la suma `(n1 + n2 + n3)` y forzar a que se resuelva antes de dividir entre 3.

